Film & Kino Agent

Sitzung 02a – Retrieval Strategien

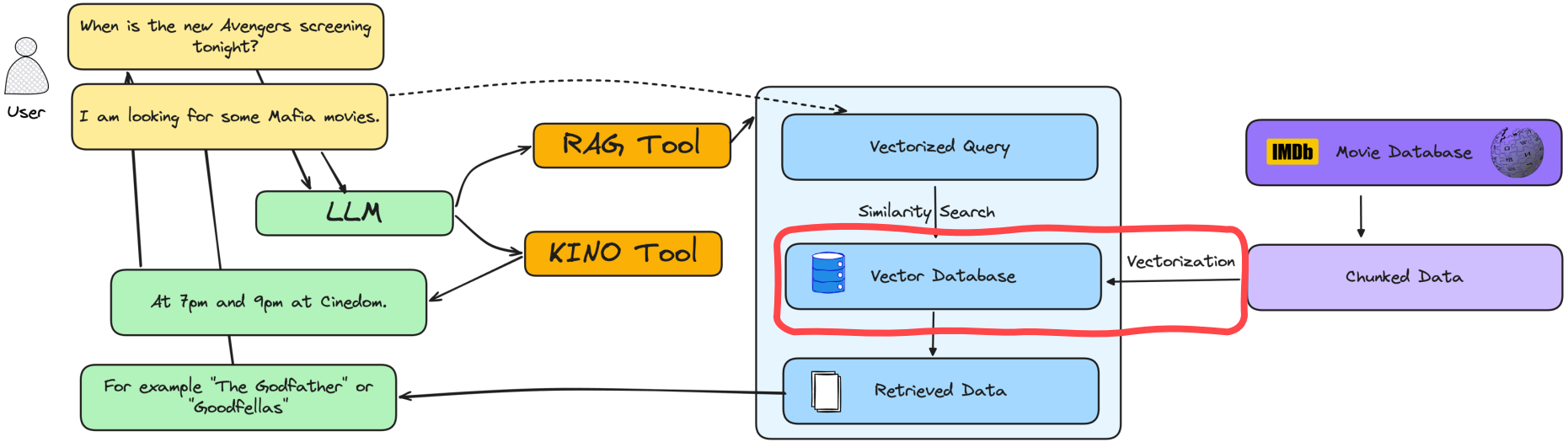
Prof. Dr. Richard Sieg
SoSe 2026

DIS22a

Termine

Datum	Inhalt
15.4	Kick-Off, Projektvorstellung, Setup
22.4	UI bauen, LLM-API-Anbindung, Prompt Engineering
29.4	Embeddings & Vektordatenbanken
6.5	Filmdatenbank vektorisieren
13.5	*Home Office*
20.5	RAG: Theorie + Implementation (Projektwoche)
27.5	*Home Office*
3.6	Tool Use & Agenten
10.6	Agenten-Implementation + Theorie: Evaluation
17.6	Gegenseitiger Nutzertest + Feinschliff
24.6	Feinschliff, Bugfixes
1.7	Abschlusspräsentationen

Architektur



Wo Dense Retrieval an Grenzen stößt

Das funktioniert gut (semantisch)

- "Ein Film über Einsamkeit im Weltraum" → Interstellar, Gravity, Moon
- "Gangsterfilm in New York" → The Godfather, Goodfellas

Das funktioniert schlecht (exakt)

- "Interstellar 2014" → findet ggf. Gravity statt Interstellar
- "Filme mit Timothée Chalamet" → Name wird zu allgemeinem Vektor komprimiert
- "Avatar: The Way of Water" → exakter Titel geht in der Semantik verloren

Drei Retrieval-Ansätze

Query und Dokument werden durch dasselbe Model geschickt

	Sparse (Keyword)	Dense (Semantisch)	Hybrid
Prinzip	Exakter Wort-Match	Vektor-Ähnlichkeit	Beides kombiniert
Methode	BM25	Bi-Encoder + Cosine	Dense + Sparse + Fusion
Stark bei	Titel, Namen, Jahreszahlen	Umschreibungen, Synonyme	Allen Query-Typen
Schwach bei	Synonyme, Umschreibungen	Exakte Terme	Mehr Komplexität
Filmbeispiel	"Interstellar 2014" ✓	"einsamer Astronaut im All" ✓	Beides ✓

Recap: BM25

1. Term Frequency (TF)

- Wie oft kommt der Suchbegriff im Dokument vor? → Mehr = relevanter, aber mit Sättigung (ab einem Punkt bringt Wiederholung weniger)

2. Inverse Document Frequency (IDF)

- Wie selten ist der Begriff insgesamt? → "the" hat niedrigen IDF (überall), "Interstellar" hat hohen IDF (selten, also informativ)

3. Document Length Normalization

- Längere Texte haben mehr Wörter → normalisieren, damit kurze Dokumente nicht benachteiligt werden

$$Score(D,Q) = \sum_{i=1}^{n} idf\,(\mathrm{q_i}) \cdot \frac{f(\mathrm{q_i}, D) \cdot (k+1)}{f(\mathrm{q_i}, D) + k \cdot \left(1 - b + b \cdot \left(\frac{|D|}{\mathrm{avgdl}}\right)\right)}$$

Wie kombiniert man beides?

Hybrid Search mit Reciprocal Rank Fusion (RRF)

Das Problem:

- Dense Score Werte zwischen -1 und 1
- BM25 Score: Werte zwischen 0 und 30+

→ Scores sind nicht direkt vergleichbar!

Film	BM25-Rang	Dense-Rang	RRF-Score
Interstellar	1	3	1/61 + 1/63 = 0.032
Gravity	5	1	1/65 + 1/61 = 0.032
The Martian	3	4	1/63 + 1/64 = 0.031
Apollo 13	2	8	1/62 + 1/68 = 0.031

Die Lösung: Nur die Ränge nutzen, nicht die Scores:

$$\mathrm{RBF}(d) = \sum {}^{1}\!\big/_{(k + rank(d))}, k = 60 \text{ (default)}$$

Drei Arten, Query und Dokument zu vergleichen

Bi-Encoder

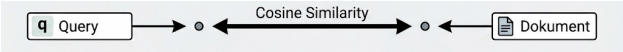
- Query und Dokument werden getrennt eingebettet → je ein Vektor → Cosine Similarity.
- Schnell, weil Dokumente vorab berechnet werden.
- Aber: ein ganzer Text wird in einen einzigen Vektor komprimiert.

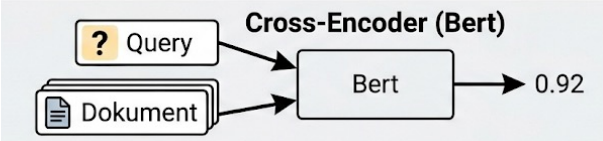
Cross-Encoder

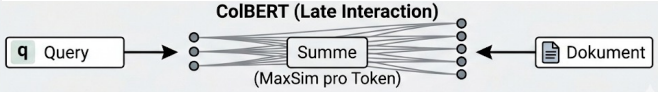
- Query und Dokument werden zusammen in ein Modell gegeben → ein Relevanz-Score.
- Genauer, weil das Modell die Interaktion zwischen Query und Dokument sieht.
- Aber: muss für jedes Query-Dokument-Paar neu berechnet werden → viel zu langsam für tausende Dokumente.

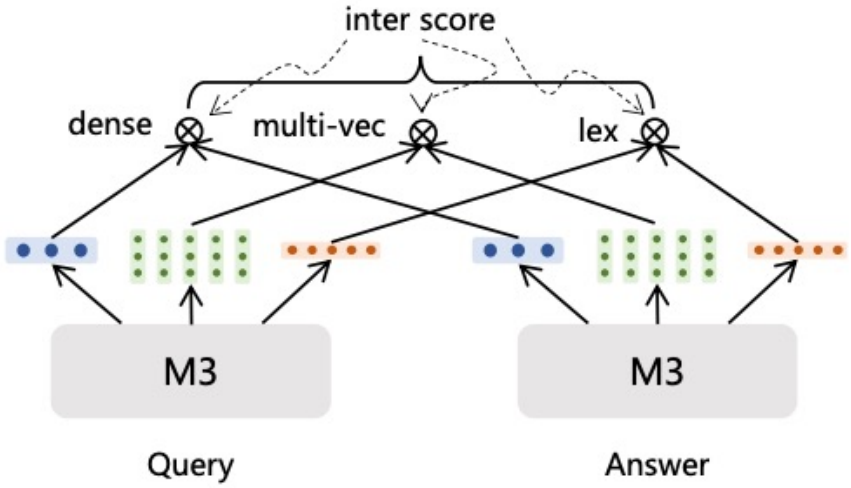
ColBERT

- Query und Dokument werden getrennt eingebettet, aber statt einem Vektor pro Text werden alle Token-Vektoren behalten.
- Verglichen wird auf Token-Ebene: für jedes Query-Token das ähnlichste Dokument-Token finden (MaxSim), dann aufsummieren.







BGE-M3: Dense + Sparse + ColBERT in einem Modell

Dense	Sparse	Multi-Vector
CLS-Vektor (1024d)	Token-Gewichte	Alle Token-Vektoren
Semantische Suche	Keyword-Suche	Feingranulares Matching
"Film über Einsamkeit"	"Interstellar 2014"	Kombination